

Wireless access to the Melexis IO board

Status: draft for discussion. Two workable approaches are described below, with what each costs and what is still unverified. Nothing here requires a change to the web tools.

The constraint

The web tools reach the board through the **Web Serial API**, which needs a serial port on the host machine. The board itself enumerates as a **USB CDC device** — it is a USB *device*, not a UART, so a Bluetooth module cannot simply be attached to the USB connector. Any wireless link therefore has to end in something the host can open as a serial port.

Two things make this easier than it sounds:

- The firmware already runs a **second SCPI console on a UART**, alongside the USB one. A dedicated task creates its own terminal instance on the UART file descriptors, and the console is compiled onto **USART2** in interrupt mode. No firmware change is needed to talk to the board over a wire other than USB.
- Note that the `:UART3:` and `:UART4:` commands are *instrument* UARTs — peripherals the board drives on request. They are not the console, and should not be confused with it.

Option A — Bluetooth Classic SPP on the UART console

A Serial Port Profile module is wired to the board's USART2 console pins. The workstation pairs with it once; `rfcomm bind` then presents it as `/dev/rfcomm0`, which is a serial port like any other.

What is needed: an SPP module, wiring to the console pins, and a supply for the module. No extra computer, no firmware change, no change to the web tools.

Advantages

- The simplest topology: board, module, workstation. Nothing in between.
- Once paired, connecting is a single command on the workstation, and can be made permanent.
- No root privileges for day-to-day use after the initial bind.
- The board stays a normal instrument; the USB console remains available in parallel.

Disadvantages

- **Bandwidth.** Bluetooth Classic SPP realistically carries 100–300 kbit/s. One MLX90640 frame is 1668 bytes, which the board returns as comma-separated hex — roughly 5 KB of text per frame. That is about a fifth of a second per frame at best, before the I2C transfer itself is counted. Fine for the MLX90632 and MLX90614, painful for the thermal arrays.
- **Line settings now matter.** Over USB CDC the baud rate never reaches the device and is ignored; over a real UART it must match the module at both ends. The terminal page keeps those controls

under *Line settings* for exactly this case.

- The console's default baud rate on USART2 has to be confirmed before configuring the module.

Unverified: whether Chrome lists `/dev/rfcomm0` in the Web Serial device picker. Chrome filters candidate serial devices by udev properties, and `rfcomm` nodes do not always carry them. This decides whether the approach works at all, and it takes a minute to test once a module is available.

Option B — USB/IP over an IP link

A small computer next to the board exports the USB device over TCP; the workstation imports it. The remote kernel stops driving the device and hands it to `usbipd`; locally, the `vhci-hcd` module presents a virtual USB port, the device enumerates again, and `/dev/ttyACM0` appears as if the cable were plugged in directly.

What is needed: a small host at the board (a Pi-class machine), an IP link between the two, and `usbip` on both sides. The workstation already has the tooling — `usbip-utils 2.0` and the `vhci-hcd`, `usbip-host` and `usbip-core` modules.

Board side:

```
sudo modprobe usbip-host
sudo usbipd -D
usbip list -l           # find the bus id, e.g. 1-1.2
sudo usbip bind -b 1-1.2
```

Workstation side:

```
sudo modprobe vhci-hcd
usbip list -r <host>
sudo usbip attach -r <host> -b 1-1.2
```

`usbip port` lists what is attached and `sudo usbip detach -p 00` releases it.

Advantages

- The board appears exactly as it does over USB, so every tool behaves identically — including the DFU updater and anything else that expects a CDC device.
- Bandwidth is whatever the network gives. The board is a full-speed 12 Mbit device, so Wi-Fi is never the bottleneck and the thermal arrays stay usable.
- Works over any IP transport, including Bluetooth PAN if that is the only option — though the link then becomes the bottleneck again.

Disadvantages

- A second computer to buy, power, configure and keep running.

- Root on both sides for every attach and detach, so practical use means a systemd service.
- **Traffic on port 3240 is unencrypted and unauthenticated.** On any untrusted network it has to be tunnelled, for example `ssh -L 3240:localhost:3240 user@host` and then attaching to `localhost`.
- More moving parts to fail, and the failure modes are less obvious than a cable.

Comparison

	Bluetooth Classic SPP	USB/IP
extra hardware	SPP module	small computer
firmware change	none	none
change to the web tools	none	none
throughput	100–300 kbit/s	link speed; not the bottleneck
thermal arrays usable	poorly	yes
privileges in daily use	none after pairing	root, or a service
security	Bluetooth pairing	plaintext unless tunnelled
unknowns	does Chrome list <code>/dev/rfcomm0</code>	none of substance

Recommendation

Test Option A first. It is cheaper, simpler and sufficient for the single-zone sensors, and the one thing that could rule it out is answerable in minutes. If Chrome does not enumerate the `rfcomm` node, or if the thermal arrays need to be usable wirelessly, Option B is the fallback and is known to work.

Open questions

1. Does Chrome's Web Serial picker list `/dev/rfcomm0` ? Blocks Option A entirely.
2. What baud rate is the USART2 console initialised at, and does it match a candidate module?
3. Is wireless needed for the thermal arrays, or only for the single-zone sensors? This decides whether the bandwidth ceiling of Option A matters.